

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Створення інтегрованих середовищ розробки»

Робота №3

Тема «Семантичний аналізатор»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Дейнега Л.Ю.

2025

МЕТА

Метою роботи є ознайомитись з основними способами, засобами та методами, які використовуються під час семантичного аналізу вихідної програми на основі абстрактних синтаксичних дерев; навчитися проєктувати та розробляти семантичний аналізатор у складі компілятора.

ЗАВДАННЯ

Завдання на роботу:

1. Ознайомитись з теоретичними відомостями, необхідними для виконання роботи.
2. Програмно реалізувати формування таблиці символів.
3. Програмно реалізувати створення дерев проміжного представлення (вручну, додаючи відповідну функціональність у програму, або використовуючи в генераторі парсерів ANTLR при побудові граматики відповідної дії).
4. Реалізувати виконання програмним забезпеченням перевірки зокрема наступних правил (у відповідності з індивідуальним завданням з додатку А, узгодженим з викладачем) та генерацію відповідних інформативних повідомлень (файл програми, номер рядка, позиція в рядку, текст повідомлення) у разі виявлення помилок:
 - жодний ідентифікатор (змінна, атрибут, функція, або метод) не оголошено двічі в одній області видимості;
 - жодний ідентифікатор (змінна, атрибут, функція або метод) не використано раніше ніж його оголошено в даній області видимості або без оголошення у ній;
 - кількість та тип фактичних аргументів у виклику функції (методу) має бути такою ж як формальних параметрів у визначенні функції;

- при поверненні результати функцією значення має бути того ж типу, що і в оголошенні функції;
- методи та функції, які включені у вирази, мають повертати значення;
- керуючі оператори (break, continue) мають входити у тіло циклу;
- вираз, включений в умовний оператор, має бути логічного типу або перетворюватися у нього (у відповідності з правилами мови програмування);
- операнди операцій мають відповідати типам, для яких визначені дані операції;
- індекс масиву має бути цілочисельним;
- функції без типу не мають повертати значення;
- ієрархія класів (за умов їх присутності в граматиці мови) вірно побудована;
- зарезервовані ключові слова використано вірно.

Семантичні перевірки повинні виконуватися згори вниз. Тільки перевірка типів у складі семантичних перевірок може виконуватись у протилежному напрямку.

Усі виявлені помилки мають виводитись за один запуск.

5. Розширити семантичний аналіз виведенням попереджень у разі виявлення наступних умов:

- оголошена змінна не використовується жодного разу в програмі;
- оператори циклу не передбачають завершення циклу;
- умовні оператори не передбачають можливість руху всіма наявними своїми гілками.

Усі виявлені попередження мають виводитись за один запуск.

6. Відповідним чином модифікувати інтерфейс програмного забезпечення для виведення результатів виконання семантичного аналізу (пункти 2-5).

7. Виконати тестування побудованого синтаксичного аналізатора.
8. Оформити звіт.
9. Відповісти на контрольні питання.

ВИКОНАННЯ

Код

RustIDE

```
import gen.MainLexer;
import gen.MainParser;
import org.antlr.v4.runtime.CharStreams;
import org.antlr.v4.runtime.CommonTokenStream;
import org.antlr.v4.runtime.tree.ParseTree;
import org.antlr.v4.runtime.tree.ParseTreeWalker;

import javax.swing.*;
import java.awt.*;

public class RustIDE extends JFrame {
    private JTextArea codeArea = new JTextArea();
    private JTextArea outputArea = new JTextArea();

    public RustIDE() {
        setTitle("Rust Semantic Analyzer IDE");
        setSize(800, 600);
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        JButton btn = new JButton("Run Full Analysis");
        btn.addActionListener(e -> analyze());

        codeArea.setFont(new Font("Consolas", Font.PLAIN, 14));

        add(new JScrollPane(codeArea), BorderLayout.CENTER);
        add(new JScrollPane(outputArea), BorderLayout.SOUTH);
        add(btn, BorderLayout.NORTH);
        outputArea.setPreferredSize(new Dimension(800, 200));
        outputArea.setEditable(false);
    }

    private void analyze() {
        outputArea.setText("");
        MainLexer lexer = new MainLexer(CharStreams.fromString(codeArea.getText()));
        MainParser parser = new MainParser(new CommonTokenStream(lexer));
        ParseTree tree = parser.program();

        ParseTreeWalker walker = new ParseTreeWalker();
        SemanticAnalyzer analyzer = new SemanticAnalyzer();
        walker.walk(analyzer, tree);
    }
}
```

```

        if (analyzer.errors.isEmpty() && analyzer.warnings.isEmpty()) {
            outputArea.setText("Analysis complete. No issues found.");
        } else {
            for (String err : analyzer.errors) outputArea.append(err + "\n");
            for (String warn : analyzer.warnings) outputArea.append(warn +
"\n");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(()-> new RustIDE().setVisible(true));
    }
}

```

SemanticAnalyzer

```

import org.antlr.v4.runtime.Token;
import org.antlr.v4.runtime.tree.ParseTreeProperty;

import java.util.ArrayList;
import java.util.List;

import gen.*;

public class SemanticAnalyzer extends MainBaseListener {
    private Scope currentScope = new Scope(null);
    private int loopDepth = 0;
    public List<String> errors = new ArrayList<>();
    public List<String> warnings = new ArrayList<>();
    private ParseTreeProperty<Type> types = new ParseTreeProperty<>();

    private void reportError(Token t, String msg) {
        errors.add("ERROR ["+t.getLine()+":"+t.getCharPositionInLine()+"]: "+msg);
    }

    @Override public void enterBlock(MainParser.BlockContext context) {
        currentScope = new Scope(currentScope);
    }

    @Override public void exitBlock(MainParser.BlockContext context) {
        // Перевірка на невикористані змінні перед виходом
        for (Symbol s : currentScope.getSymbols().values()) {
            if (!s.isUsed && !s.isFunction) {
                warnings.add("WARNING ["+s.line()+":"+s.charPos+"]: Variable '"+s.name+"' is
never used.");
            }
        }
        currentScope = currentScope.getEnclosingScope();
    }

    @Override public void enterLetBinding(MainParser.LetBindingContext context) {
        String name = context.ID().getText();
        if (currentScope.resolveLocal(name) != null) {

```

```

        reportError(context.ID().getSymbol(), "Identifier '"+name+"'
already declared in this scope.");
    }
    Type t = context.type() != null ?
Type.valueOf(context.type().getText().toUpperCase()) : Type.UNKNOWN;
    currentScope.define(new Symbol(name, t,
context.ID().getSymbol().getLine(),
context.ID().getSymbol().getCharPositionInLine()));
    }

    @Override public void enterIdExpr(MainParser.IdExprContext context) {
        String name = context.ID().getText();
        Symbol s = currentScope.resolve(name);
        if (s==null) {
            reportError(context.ID().getSymbol(), "Variable '"+name+"' used
without declaration");
        } else {
            s.isUsed = true;
            types.put(context, s.type);
        }
    }

    @Override public void enterLoopStatement(MainParser.LoopStatementContext
context) { loopDepth++; }
    @Override public void exitLoopStatement(MainParser.LoopStatementContext
context) { loopDepth--; }

    @Override public void enterBreakStmt(MainParser.BreakStmtContext context)
{
        if (loopDepth==0) reportError(context.getStart(), "'break' used
outside of loop.");
    }

    @Override public void exitIndexExpr(MainParser.IndexExprContext context) {
        Type idxType = types.get(context.expression(1));
        if (idxType != Type.I32) {
            reportError(context.getStart(), "Array index must be an integer
(i32).");
        }
    }
}

```

SymbolTable

```

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

enum Type {I32, F64, BOOL, STRING, VECTOR, VOID, UNKNOWN}
class Symbol {
    String name;
    Type type;
}

```

```

    int line, charPos;
    boolean isUsed = false;
    boolean isFunction = false;
    List<Type> paramTypes = new ArrayList<>();

    public Symbol(String name, Type type, int line, int charPos) {
        this.name = name;
        this.type = type;
        this.line = line;
        this.charPos = charPos;
    }
}

class Scope {
    private Scope parent;
    private Map<String, Symbol> symbols = new HashMap<>();

    public Scope(Scope parent) { this.parent = parent; }

    public void define(Symbol sym) { symbols.put(sym.name, sym); }

    public Symbol resolve(String name) {
        Symbol s = symbols.get(name);
        if (s != null) return s;
        if (parent != null) return parent.resolve(name);
        return null;
    }

    public Symbol resolveLocal(String name) { return symbols.get(name); }
    public Scope getEnclosingScope() { return this.parent; }
    public Map<String, Symbol> getSymbols() { return symbols; }
}

```

Результати

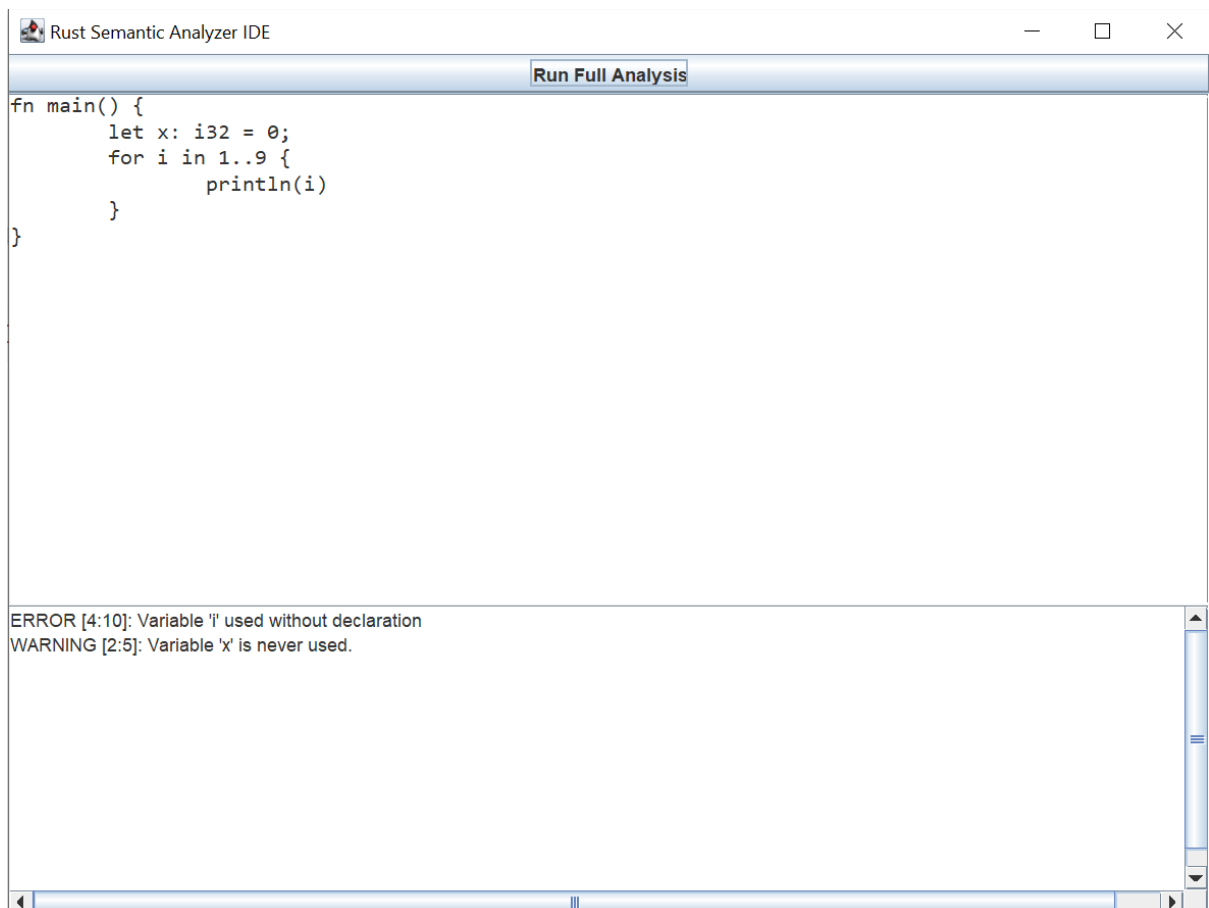


Рисунок 1.1 — Вигляд програми

ПИТАННЯ

Які завдання виконуються на етапі семантичного аналізу?

Наступні задачі вирішуються на етапі семантичного аналізу:

- перевірка типів;
- контроль потоку керування;
- аналіз функцій.

Яким чином виконується перевірка типів?

Листя дерева: визначаються тип константи, наприклад 5 це i32, "string" це String.

Підйом вгору: для кожного вузла аналізатор перевіряє типи його дітей.

Що таке таблиця символів та яка її структура?

Таблиця символів — це службова структура даних, яка зберігає всю інформацію про ідентифікатори в програмі.